From Prototype to Enterprise Gen AI Assistant

One Portable Codebase, Two Deployment Targets

GCP-Managed (default) vs No-Lock-In On-Prem

Decision-grade comparison for enterprise architects and the exec sponsor.

All figures are order-of-magnitude planning numbers, not vendor quotes.

Agenda

Part 1 — Overview. What we're building, the architecture, two topologies, framing assumption, locked decisions.

Part 2 — Deep Dive. Dimension by dimension: time-to-market, quality, cost/TCO, risk, security, sovereignty, scalability/ops.

Part 3 — The Decision. Weighted scorecard, sovereignty gate, risk register, where on-prem wins, rebuttal, portability caveats, roadmap, recommendation.

Bottom Line Up Front

- **Build ONCE, deploy two ways.** One codebase; only adapters and the platform beneath them change.
- **Default to GCP-managed:** fastest TTM (~5× less assembly), best quality, lowest TCO (~35–40% of on-prem). Scorecard **4.67 vs 2.57 out of 5**.
- **Build on-prem as insurance:** the no-lock-in / data-residency option; decisive only under a sovereignty mandate or already-sunk platform + people.
- **Not a one-way door:** portability is an architectural property of the hexagonal CORE, not a deployment choice.

Part 1 of 3

Overview (high level)

Executive Summary

- **Build ONCE, deploy two ways.** Hexagonal (ports-and-adapters) design keeps a provider-agnostic CORE identical across both targets. Only the adapters and platform change.
- **Default to GCP-managed.** Best quality (Agent Search on Gemini Enterprise Agent Platform + Document AI + Gemini Flash), fastest TTM (~7 vs ~36 eng-weeks, ~5×), lowest TCO (~$2.1M vs ~$5.7M over 3 yrs).
- **TCO gap is people, not gear.** Inference is external in both. On-prem: ~4–6 FTE platform ops; GCP: ~1–1.5 FTE to integrate, not operate.
- **Honest baseline.** Year-0 ports-and-adapters refactor is costed once in both columns — that's why Year-0 is non-zero on both sides.
- **Build on-prem to a tested baseline as insurance.** No-lock-in / data-residency / air-gap option; competitive only when sovereignty mandates it or capacity is already sunk.

What We Are Building

An **enterprise Gen AI document assistant**: upload documents, find what you're allowed to see, and chat over your files with cited answers.

Capability	What it does
Ingest	Upload PDF/text; parse, structure-aware chunk, index on arrival
Access-aware search	Hybrid keyword + semantic, ABAC-filtered; restricted matches show a redacted card
RAG chat with citations	Grounded only in docs the user may see; inline [n] citations; Validator groundedness gate
Access-request workflow	One-click request, approver email, time-limited grant
Audit	Append-only log of every auth, search, view, and AI turn

Security invariant: access enforced server-side on EVERY retrieval — never in the UI.

Architecture at a Glance: One Core, Swappable Adapters

Ports-and-adapters (hexagonal). The product is written once; the cloud underneath is a config choice.

Layer	Content	Changes per target?
CORE	ABAC, chunking, retrieval, ingest, audit; agent graph; /api routes; UI	No (byte-identical)
PORTS	LLM, Embedder, Reranker, Retriever, Store, ObjectStore, Parser, Guardrail, DLP, EventBus, Telemetry, Identity, Orchestrator	No (interfaces)
ADAPTERS	One implementation set per target	Yes
PROFILE	`UNLOCK_PROFILE` = local / onprem / gcp selects the adapter set	The one switch

- **~80% of the code never changes** between targets (CORE + UI + eval + tests).
- What differs: only `adapters/<target>/`, `profiles/<target>.yaml`, `deploy/<target>/`.

Deployment at a Glance: Two Topologies

Same request path; different platform beneath the ports. Inference is an external hosted endpoint in **both**.

GCP-managed: Apigee + Cloud Armor → Cloud Run (CORE) → Agent Runtime/ADK → Agent Search on Gemini Enterprise Agent Platform + Document AI over AlloyDB + GCS; Model Armor, Cloud DLP, Eventarc, Cloud Observability.

No-lock-in on K8s: Kong/Envoy + OIDC → CORE pod → ADK on K8s → OpenSearch + bge + Tika over pgvector + MinIO; Llama Guard + NeMo + Presidio, Knative/KEDA, OTel + Grafana.

Tier	GCP	On-prem
Gateway	Apigee X + Cloud Armor	Kong / Envoy + OIDC
Agent runtime	Agent Runtime on Gemini Enterprise Agent Platform + ADK	ADK on K8s
Retrieval	Agent Search on Gemini Enterprise Agent Platform + Document AI	OpenSearch + bge + Tika
Data	AlloyDB + GCS	pgvector + MinIO
Safety / DLP	Model Armor + Cloud DLP	Llama Guard + NeMo + Presidio

Framing Assumption: Inference Is External in BOTH Targets

> **No AI inference runs on-premise in either target.** Both call a hosted model endpoint (GCP: Gemini on Gemini Enterprise Agent Platform; on-prem: Gemma via LiteLLM). The comparison is the PLATFORM around the model.

Implication	Consequence
Model inference $ is equal both sides	Excluded from the differential TCO
No inference GPU on either side	On-prem gets no credit for "avoiding GPU spend"
Single model port is the seam	Swapping Gemini for Gemma is one adapter line, not a CORE change

The comparison is purely the platform: gateway, guardrails, agent runtime, retrieval, parsing, DB, storage, eventing, DLP, observability, and the people who run it.

Scope and Assumptions

Area	Assumption
Workload	~500–2,000 internal users; ~1M docs; moderate bursty volume; ~99.9% target
Cost basis	~$160k–$280k/FTE loaded; managed services as usage-based opex
Inference	External hosted endpoint in both; cost equal, excluded from differential
In scope	Platform: gateway, agent runtime, retrieval, parsing, DB, storage, eventing, DLP, safety, observability + people
Out of scope	Model fine-tuning, GPU capex/ops, data-migration specifics, org compliance audits
Vendor figures	Context window, durability, infoType counts are vendor-cited as of 2026: verify at procurement

Locked Architecture Decisions

#	Decision (final)
1	**Agent layer = ADK in both.** GCP: Agent Runtime on Gemini Enterprise Agent Platform + ADK. On-prem: same ADK app on K8s. Agent graph + prompts live in CORE and are reused; only model binding and host differ.
2	**Retrieval.** GCP: Agent Search on Gemini Enterprise Agent Platform + Document AI; AlloyDB holds chunk text + ABAC side-tables. On-prem: OpenSearch (BM25+kNN) + bge-reranker; pgvector holds chunk text + ABAC.
3	**Validator / groundedness gate** runs before every answer, in both targets.
4	**ABAC is CORE-owned (`AccessPredicate`), compiled per backend:** SQL (SQLite/pgvector/AlloyDB), filter-DSL (Agent Search), DLS (OpenSearch). Policy model + orchestration reused; enforcement per-adapter.
5	**Embedder and Reranker are first-class ports.** On-prem depends on 4 external hosted inference endpoints (LLM, embedder, reranker, safety). GCP folds embedding + reranking into managed Agent Search on Gemini Enterprise Agent Platform.

Scaffold Reality (Honest Current State)

Profile	State today	What it proves
Prototype	SQLite + local FS + Anthropic API. No Postgres, no pgvector, no ADK graph, no port boundary yet.	Domain logic (ABAC, chunking, ACL retrieval, citations, audit) works end-to-end.
`local`	Runs end-to-end (lightweight in-process runner).	CORE prompts and Orchestrator graph are real and runnable.
`onprem`	Code-complete (docker-compose).	Four hosted-inference ports and OSS adapters are wired, not aspirational.
`gcp`	Real adapter code; requires a GCP project to run.	Gemini Enterprise Agent Platform / AlloyDB / Document AI adapters exist in code.

- **The ports-and-adapters refactor is the shared Year-0 build**, common to both targets. It nets out of the differential and explains the non-zero Year-0 on both sides.
- Reuse/portability claims are **designed intent realized in this repo**, not a claim that the GCP profile is in production.

Side-by-Side Stack Mapping

Everything above the ports ships unchanged (ABAC, chunking, citations, Validator, audit, UI). Only the rows below differ.

CORE port	GCP-managed	On-prem
API gateway / edge	Apigee X + Cloud Armor	Kong / Envoy + ModSecurity
Safety / guardrails	Model Armor	Llama Guard 4 + NeMo + Presidio
Model / generation	Gemini	Gemma (LiteLLM)
Agent runtime	Agent Runtime + ADK (managed)	Same ADK app on K8s
Retrieval + rerank	Agent Search (RRF + reranker)	OpenSearch (BM25+kNN) + bge-reranker
Doc parsing / OCR	Document AI Layout Parser	Tika / Unstructured + Tesseract
PII / DLP	Cloud DLP	Presidio
Data (object + vector)	GCS + AlloyDB (ScaNN/pgvector)	MinIO + PostgreSQL/pgvector
Eventing + observability	Eventarc + Cloud Observability	Knative/KEDA + OTel/Prometheus/Grafana

Stack Mapping: The Reuse Win

- **Embedder + Reranker are ports, not buried config.** On-prem must host them as separate inference endpoints; GCP folds both into managed Agent Search on Gemini Enterprise Agent Platform.
- **On-prem depends on 4 external hosted inference endpoints** (LLM, embedder, reranker, safety). GCP calls one model endpoint and bundles the rest.
- **One Postgres-compatible repository** serves both targets (AlloyDB is Postgres-wire-compatible). Same model port, same ADK agent graph, same ABAC policy model.
- **Many "on-prem vs GCP" choices collapse to a connection string** + one-line index strategy (`USING scann` vs `USING hnsw`) + per-backend ABAC compiler.
- **ABAC nuance:** policy model + orchestration reused; enforcement compiled per adapter. NOT identical SQL everywhere.

Part 2 of 3

Deep Dive: Dimension by Dimension

Dimension: Time-to-Market

Platform assembly + hardening only (inference external both ways; CORE identical). Deltas are relative effort bars.

Capability	GCP	On-prem	Edge
API gateway / authz edge	light	heavy	**GCP** — Kong HA, plugin tuning, cert rotation are DIY
Guardrails / injection	light	heavy	**GCP** — NeMo rails authored/tested; host Llama Guard
Retrieval / index / rerank	light	heaviest	**GCP** — OpenSearch sizing + separate reranker + fusion
Doc parsing / layout	light	heavy	**GCP** — Layout/OCR hand-tuned; scale the parser yourself
PII / redaction	light	heavy	**GCP** — Presidio recognizers, custom entities, FP/FN tuning
Event-trigger ingestion	light	heavy	**GCP** — Event mesh, scale-to-zero, retry/DLQ self-built
Object store	light	medium	**GCP** — Distributed MinIO, erasure coding, lifecycle
Structured + vector DB	light	heavy	**GCP** — Self-run HA Postgres: replication, failover, PITR
Eval / observability	light	heaviest	**GCP** — Four OSS tools to deploy, wire, dashboard, upgrade
Cross-cutting security	light	heavy	**GCP** — More seams → IAM, network policy, pen-test
Total (assembly to prod)	**~7 eng-weeks**	**~36 eng-weeks**	**GCP ~5× faster**

Time-to-Market: Milestones

Milestone	GCP	On-prem	Edge
First working slice	~2 weeks	~6–8 weeks	**GCP**
Production-hardened (HA, security, eval, observability)	~6–8 weeks	~5–7 months	**GCP**
Ongoing platform ops (recurring)	low (config)	~0.5–1.0 FTE steady-state	**GCP**

- TTM total (~7 vs ~36 eng-weeks) is **one-time assembly only**. The ~0.5–1.0 FTE is the separate recurring ops line, not double-counted.
- The shared CORE refactor is the Year-0 build common to both; the TTM **delta** is platform assembly, not the CORE.
- On-prem also carries a recurring patch/upgrade tax across ~10 OSS systems plus four external inference endpoints.

Dimension: Quality

Sub-dimension	GCP-managed	On-prem OSS	Edge
Retrieval + parsing	Agent Search (RRF + reranker) + Document AI	OpenSearch BM25/kNN + bge; Tika	**GCP**
Model / generation	Gemini: ~1M context, strong grounding	Gemma: trails on long-context	**GCP**
Safety (guardrails + PII)	Model Armor + Cloud DLP, managed	Llama Guard + NeMo + Presidio, self-run	**GCP**
Eval tooling	Agent Platform Evals: turnkey	RAGAS + Langfuse + Phoenix: DIY	**GCP**

GCP leads every sub-dimension; eval tooling only slightly.

Quality: The Honest Read

- **Where GCP clearly wins:** reranker + layout-parser combo raises the retrieval floor with zero tuning; Gemini > Gemma on grounded generation; guardrail/DLP coverage stays current without rail-authoring labor.
- **Honest counter-point:** a well-tuned OpenSearch + bge-reranker-v2-m3 is genuinely strong; the retrieval gap narrows once tuned. OSS eval (RAGAS/Langfuse/Phoenix) is legitimately excellent.
- **The durable gap is in the MODEL (Gemini vs Gemma) and the zero-tuning FLOOR** — not in measurement. The model gap can't be closed by ops effort.
- **Cross-target caveat:** absolute eval scores stay backend-relative. Regression gates must use normalized metric names/ranges, not absolute numbers.

Verdict: GCP wins; gap is largest exactly where RAG answer quality is made.

Dimension: Cost / TCO

Inference excluded (equal, external both sides). Annual run-rate (infra + people).

Cost shape	GCP-managed	On-prem-on-K8s	Edge
Pricing model	Usage-based opex; scales toward zero when idle	License + 3-yr hardware amortization + people	**GCP**
HA/DR, patching, certs, upgrades	Provider-absorbed	Engineered, staffed, and drilled in-house	**GCP**
Platform/SRE headcount	~1.0–1.5 FTE (integrate + observe)	~4–6 FTE (operate substrate 24×7 + on-call)	**GCP**
Annual run-rate (ex-inference)	~$350k–$700k	~$1.1M–$2.2M	**GCP**

- Headcount assumes **net-new hires**. If an existing under-utilized platform team absorbs it, the people delta shrinks materially (see "Where On-Prem Wins"). The two assumptions are mutually exclusive.
- On-prem also runs four external inference endpoints (LLM, embedder, reranker, safety); GCP consumes one model endpoint + the managed bundle.

Cost: Why the Headcount Gap Is Real

Each on-prem role is undifferentiated heavy lifting — none differentiates the product:

On-prem role	Why required
K8s / platform engineer(s)	Cluster lifecycle, upgrades, autoscaling, add-ons, capacity
Search / DB specialist	OpenSearch shard/JVM/heap tuning; pgvector index + failover
Storage / eventing engineer	MinIO erasure-coding + rebuilds; Knative/KEDA/Argo health
Security engineer	CVE patching cadence, secrets, policy, DLP/guardrail upkeep, audit
On-call rotation	24×7 needs ≥ 4 people to be humane and sustainable

On GCP the same coverage is the provider's job, baked into per-unit pricing and shared across the fleet. On-prem pays full loaded salary and can't share it. This is what "open source is free" comparisons leave out.

3-Year TCO Summary

> Inference excluded (equal, external both sides). Headcount at loaded cost (~$160k–$280k/FTE).

Line	GCP-managed	On-prem-on-K8s
Year 0 one-time (build / migration)	$0.15M	$0.45M
Year 1 run-rate (infra + people)	$0.53M	$1.65M
Year 2 run-rate (+10% GCP usage; on-prem flat)	$0.58M	$1.68M
Year 3 run-rate	$0.63M	$1.72M
3-year TCO (ex-inference)	**~$1.9M**	**~$5.5M**
+ shared inference (~$60k/yr × 3, equal both)	+$0.18M	+$0.18M
3-year fully-loaded TCO	**~$2.1M**	**~$5.7M**

- **GCP lands at ~35–40% of on-prem 3-year TCO.** Gap dominated by people (~$0.75M cumulative on GCP vs ~$3.6M on-prem).
- Year-0 delta is platform assembly; the shared CORE refactor is equal in both.

Dimension: Risk (Net Posture)

Net = exposure after typical mitigations.

Risk factor	GCP-managed	On-prem OSS	Edge
Operational (systems to run/scale/fail over)	Low (managed)	High (~10 on-call surfaces + 4 inference endpoints)	**GCP**
Security-patch lag (CVEs)	Low (provider-side)	High (you own CVE-to-patch SLA across the stack)	**GCP**
Talent / key-person	Low-Med (common skills)	High (scarce OpenSearch + PG-HA + K8s-eventing depth)	**GCP**
Supply-chain (deps / images)	Low (managed surfaces)	High (transitive deps across ~10 projects)	**GCP**
Cost overrun	Med (needs budgets/quotas/alerts)	Med (pre-provision peak; step-function)	Tie
Data residency / sovereignty	Low-Med (region pin, provider-held)	Low (full physical control)	**On-prem**

Dimension: Security & Compliance

Control	GCP-managed	On-prem OSS	Edge
Encryption / keys	CMEK via Cloud KMS, auto-rotation	DIY KMS / Vault + KES	**GCP**
Network isolation	VPC-SC perimeter, IAM, private endpoints	NetworkPolicy + mTLS (SPIFFE/SPIRE), self-built	**GCP**
Certifications	Inherited (Assured Workloads)	Assemble + attest yourself	**GCP**
Edge protection	Cloud Armor WAF + L7 DDoS at Google edge	ModSecurity/Coraza WAF; DDoS solved elsewhere	**GCP**
Audit logging	Cloud Audit Logs feed CORE audit	OTel/Prom/Grafana, more wiring	**GCP**
ABAC (server-side)	CORE model → Agent Search filter-DSL	CORE model → OpenSearch DLS / SQL	Tie
Dev-auth `X-User` in prod	Apigee strips client identity headers	Gateway strips; trust signed/mTLS only	Tie

Dimension: Data Residency / Sovereignty

Factor	GCP-managed	On-prem OSS	Edge
Physical location of data	Region-pinned, in provider infrastructure	Fully in customer DC/cluster; control plane can be air-gapped	**On-prem**
"Data never leaves org boundary" mandate	Region pin + VPC-SC + CMEK; ultimate control is the cloud's	Full control; satisfies the strictest mandates	**On-prem**
Inference egress (both targets)	External model call leaves the boundary	External model call leaves the boundary	Tie (by assumption)
Control-plane air-gap	Not possible (managed control plane)	Possible for the platform (model call still external)	**On-prem**

> **Key caveat:** a true air-gap forbidding external model calls breaks **both** designs equally and needs a different inference story — a model hosted inside the sovereign boundary.

Dimensions: Scalability & Ops

Dimension	GCP-managed	On-prem OSS	Edge
Elasticity / capacity	Autoscale + scale-to-zero; usage-based, smooth	KEDA/Knative help, but reserved platform nodes wasteful when idle	**GCP**
Durability (object store)	GCS 11-nines (multi/dual-region), strong consistency	MinIO erasure-coding matches on paper; realized = ops maturity	**GCP**
Vector-retrieval at scale	AlloyDB ScaNN (vendor-cited faster vs pgvector HNSW)	pgvector HNSW; solid, manual tuning required	**GCP**
Operational burden	None (no cluster); provider-absorbed patching	Continuous K8s + ~10 OSS + 4 inference endpoints to run/patch	**GCP**
Talent / skills	Common skills, ~1.0–1.5 FTE	Scarce depth, ~4–6 FTE	**GCP**

Part 3 of 3

The Decision

Weighted Scorecard (Transparent)

Dimension	Wt	GCP	On-prem	GCP wtd	On-prem wtd
Time-to-market	15	5	2	0.75	0.30
Quality (retrieval/guardrails/eval/model)	15	5	3	0.75	0.45
Cost / TCO (fully loaded)	15	5	2	0.75	0.30
Operational burden	12	5	2	0.60	0.24
Risk (net posture)	10	4	2	0.40	0.20
Security & compliance	8	5	3	0.40	0.24
Scalability / reliability	8	5	3	0.40	0.24
Observability / eval maturity	5	5	3	0.25	0.15
Talent / skills	5	4	2	0.20	0.10
Data residency / sovereignty	4	2	5	0.08	0.20
TOTAL	**97**			**4.58**	**2.42**

GCP 4.58 vs On-prem 2.42. Scores 1–5 (5 best); GCP leads every high-weight dimension.

Scorecard: Sovereignty as a Gate

The weighted score answers the *default* question. A hard residency mandate changes the question type.

Scenario	How sovereignty is treated	Result
No hard residency mandate	Sovereignty is one weighted factor (weight 4)	**GCP wins** (4.58 vs 2.42)
Hard residency mandate applies	Sovereignty is a **pass/fail GATE**, not a weighted factor	**On-prem wins outright** (GCP is off the table regardless of price)

- When data legally may not sit in a managed cloud, you don't weigh it — it's a gate, and only the on-prem profile clears it.
- Absent that gate, the weighted result is robust to reasonable weight changes.

> **Sensitivity check:** gap = 4.58 − 2.42 = ~2.16. Doubling sovereignty weight (4→8) shifts ~+0.08 to on-prem, leaving ~2.1 in GCP's favor. The conclusion holds.

Risk Register (After Mitigation)

On-prem

Risk	Net	Mitigation
Operational (~10 systems + 4 inference endpoints)	**High**	Hard to mitigate without heavy FTE + maturity
Security-patch lag (OSS CVEs)	**High**	Patch cadence + scanning you operate
Talent / key-person	**High**	Cross-training; skills are scarce
Supply-chain (deps / image CVEs)	**High**	SBOM + image scanning you run

GCP-managed

Risk	Net	Mitigation
Cost overrun	**Med**	Billing budgets, quotas, alerts, eval sampling
Data residency / sovereignty	**Med**	Region pin + VPC-SC + CMEK (else use the on-prem profile)
Service deprecation	**Low**	Provider lifecycle notices; adapter-isolated

Cross-cutting (both): ABAC stays in CORE; OTel is the portable telemetry seam; dev `X-User` hard-off in prod.

Honest Case: Where On-Prem Genuinely Wins

Driver	Why on-prem is the right call	Caveat
Hard data-sovereignty / residency mandate	Data + metadata legally must never leave the org boundary; region pinning is insufficient.	A constraint, not a saving. Justifies on-prem even at higher cost.
Air-gapped / classified control plane	No outbound connectivity to a public-cloud control plane permitted.	External inference is the KEY ASSUMPTION in both; a true air-gap forbidding model calls breaks both designs.
Existing idle datacenter capacity	Hardware capex is sunk; marginal cost is power + a cluster slice.	Must be genuinely idle AND adequate. "Idle but old" still fails.
Existing under-utilized platform team	Dominant cost (people) already paid with durable headroom; on-call already covers 24×7.	If you hire for this, the gap reopens immediately.
Pre-existing OpenSearch / Postgres / K8s estate	Marginal team + license cost is incremental, shared across workloads.	Only if this app is a small add to an already-funded platform.

Rule of thumb: on-prem breaks even mainly when hardware AND people are already sunk + shared, or when residency law removes GCP from contention.

Rebuttal: Why GCP Still Nets Out Ahead

On-prem argument	Rebuttal
"Open source is free"	Inference is external both ways — on-prem's only saving is moot, while it carries ~4–6 FTE ops + four inference endpoints GCP bundles. 3-yr TCO ~$2.1M vs ~$5.7M.
"We control our data"	Region pin + VPC-SC + CMEK cover most cases; for a genuine sovereignty mandate, move THAT workload to the on-prem profile — same product, no rewrite.
"We can match the quality"	Only with sustained tuning; the model gap (Gemini vs Gemma) and managed reranker + layout-parser floor can't be closed by ops.
"We can build it"	Yes — in ~5× the eng-weeks, months later, then ~0.5–1.0 FTE forever to keep ~10 OSS systems + four inference endpoints healthy.

The clincher: the same hexagonal design that makes on-prem possible is what makes GCP safe. Portability is an architectural property, not a deployment choice.

Portability Caveats (Stated Plainly)

The exit path is real but not frictionless. Honest leak points:

Area	The leak	Consequence
Object store API	GCS offers an S3-compatible API, but some advanced S3 features differ (V4 signing, generation-vs-opaque version IDs, multipart edge cases)	Small provider-specific branch in the storage adapter
ABAC enforcement	One policy model, compiled to different mechanisms (SQL / Agent Search filter-DSL / OpenSearch DLS)	Security-critical reimplementation per adapter, validated separately
Retrieval chunking	GCP may auto-chunk inside Agent Search on Gemini Enterprise Agent Platform; on-prem reuses CORE chunking	Chunk boundaries (and citation spans) not byte-identical across targets
Eval scores	Different judge models/prompts per backend	Absolute scores not comparable; gates use normalized names/ranges

Net: none of these flips the recommendation. They convert falsifiable claims into claims that survive scrutiny.

Delivery Roadmap (Phased)

Phase	Ships	Profile	Milestone
0. Carve-out	CORE + ports + composition root; local adapters	local	Full demo runs on SQLite/FTS5/pypdf (119 tests)
1. GCP data + retrieval	AlloyDB, GCS, Document AI, Agent Search, Eventarc	gcp	ABAC-filtered ingest + reranked retrieval on GCP
2. GCP generation + edge	Gemini, Apigee/IAP, Cloud Observability + Evals	gcp	Full GCP RAG behind Apigee, metrics on a dashboard
3. GCP safety	Model Armor, Cloud DLP	gcp	Injection blocked, PII redacted; GCP target complete
4. On-prem data + retrieval	pgvector, MinIO, Tika, OpenSearch, KEDA	onprem	Same demo on K8s; ABAC validated
5. On-prem gen + safety + obs	Gemma, Kong/OIDC, Llama Guard + NeMo, OTel	onprem	Both targets pass the conformance suite
6. Compare + harden	Cross-target eval, cost model, runbooks, exit-path test	both	This deck, backed by measured numbers

Final Recommendation

- **Default to GCP-managed.** Wins every high-weight dimension: fastest TTM (weeks vs quarters), best quality (reranker + layout parsing + Gemini), lowest TCO (~35–40% of on-prem), most governable risk. Scorecard **4.58 vs 2.42**.
- **BUILD on-prem to a tested baseline as insurance** for genuine trump cards: a hard sovereignty/residency mandate, air-gap control, or an existing hardened platform + ops team you're obligated to reuse.
- **Govern GCP's real risks by policy:** budgets + quotas + alerts (cost), region pin + VPC-SC + CMEK (residency).

De-risking property	Mechanism
Product never gets rewritten	One provider-agnostic CORE (ABAC, chunking, retrieval, Validator, citations, audit, UI) serves both targets.
Switching cost is bounded	Only adapters + profile change; ABAC policy, retrieval, and audit invariants are shared.
Data is portable	AlloyDB is Postgres-wire-compatible; GCS offers an S3-compatible API.
Telemetry is portable	OTel SDK in CORE; swap the exporter, not the instrumentation.
Sovereign workloads relocate later	Start on GCP for speed + quality; move a regulated workload to on-prem when a mandate demands — no product rewrite.

Bottom line: with one portable CORE, choose GCP-managed now for the fastest, highest-quality, lowest-TCO path — knowing the architecture preserves a clean exit to on-prem for any future sovereignty obligation. All figures